

ADR-0002 — Supply-chain trust: plain signing vs TUF vs Uptane (and MVP timing)

Field	Value
ADR	ADR-0002
Title	Supply-chain trust: plain signing vs TUF vs Uptane (and MVP timing)
Revision	2
Created	2026-06-07
Last modified	2026-06-07
Status	Proposed
Decision drivers	Compromise resilience of the OTA trust chain; minimal-wrapping locked strategy; mandated Go stack; Android-15-first phase; verify-before-apply requirement.
Supersedes	—
Superseded by	—
Resolves	additions_synthesis.md §5 C5 (TUF/Uptane timing); routed open question §7 ADR-0002.
Related ADRs	ADR-0001 (wrapped engine), ADR-0003 (server topology), ADR-0004 (transport).
Evidence base	../ota_landscape_report.md §3.3, §4 ; ../stacks/tuf-go-tuf.md ; ../stacks/uptane.md ; additions_synthesis.md §3, §5 C5, §6, §7 .
Anti-bluff rule	Every claim traces to a cited note/report section. Items the notes flagged as unverified are carried forward as UNVERIFIED ; this ADR introduces no facts absent from the underlying notes.
Fixed-summary (Rev 2)	Added §7 row for §11.4.123 (rock-solid-proof) marked unmappable-until-Constitution-present; re-attributed §4.1 local-verified-file decision to the verify-before-apply driver rather than

Field	Value
	the synthesis note; flagged the security/Security-KMP catalogue bricks' fit for TUF-role/threshold/offline-key primitives as UNVERIFIED in §8.

Table of Contents

1. [Context](#)
2. [Decision drivers](#)
3. [Options considered](#)
 - [3.1 Option A — Plain per-artifact signing only \(SHA-256 + detached signature + AVB\)](#)
 - [3.2 Option B — TUF via go-tuf/v2 server-side, device enforcement gated](#)
 - [3.3 Option C — Full Uptane \(dual-repo + Primary/Secondary\) now](#)
 - [3.4 Option D — aktualizr as a product](#)
4. [Decision](#)
 - [4.1 MVP trust model \(1.0.0\)](#)
 - [4.2 MVP-forward signing interface \(Uptane-ready\)](#)
 - [4.3 Adoption sequencing \(1.0.1+\)](#)
5. [Consequences](#)
 - [5.1 Positive](#)
 - [5.2 Negative](#)
6. [Status](#)
7. [Compliance notes \(HelixConstitution\)](#)
8. [Open / UNVERIFIED items](#)
9. [Sources](#)

1. Context

Helix OTA already plans per-artifact integrity for every release: **SHA-256 + a detached signature, verified server-side on upload and device-side before apply**, with the native Android A/B engine's own payload signature check as a further layer. [tuf-go-tuf §1] [additions_synthesis §3] The two operator drafts independently converged on “artifact integrity = SHA-256 + signature verification, verified server-side on upload and device-side before apply.” [additions_synthesis §3]

Draft `initial_research_02.md` went further, proposing TUF (via go-tuf/v2) inside the MVP security service. [additions_synthesis §2, §4] The locked scope, however, defers full Uptane to 1.0.1+. This is the contradiction recorded as **C5** in the reconciliation: “02 puts TUF in MVP security service; locked scope defers full Uptane to 1.0.1+.” The recorded resolution rule is: **MVP ships signed-artifact + SHA-256 + AVB; TUF/Uptane is an ADR-0002 hardening item for 1.0.1+; signing interfaces are designed MVP-forward so Uptane drops in without rework.** [additions_synthesis §5 C5] This ADR is that decision, now backed by the landscape evidence.

The trust question is distinct from the delivery question (ADR-0001). TUF/Uptane are **not** OTA delivery engines; they secure *which* artifact is authorized and *that* it is authentic/fresh, while the native A/B update_engine still performs the atomic, anti-bricking apply with automatic boot-failure rollback. [uptane §2, §9] [tuf-go-tuf §6] They are complementary, not overlapping. [tuf-go-tuf §6]

LOCKED context honored by this ADR (not re-decided here):

- Native Android A/B (AOSP update_engine) on-device + a custom Go control plane; wrap OSS only where it adds value. [ota_landscape_report front-matter, §3.1, §3.2]
- Mandated stack: Go + Gin + Brotli + HTTP/3→HTTP/2, REST-primary; PostgreSQL + MinIO/S3. [ota_landscape_report front-matter] [additions_synthesis §1]
- Phase 1 target is Android 15 first (Orange Pi 5 Max / RK3588 class); Linux/universal is a later phase. [ota_landscape_report front-matter]
- The *delivery* engine (AOSP-native-only vs hawkBit vs Mender) is decided by ADR-0001, not here. [additions_synthesis §7]

Why plain signing is not enough (the gap this ADR closes)

Plain SHA-256 + per-artifact signature covers the *wrong/arbitrary file* class **only if** the signing key is intact and the device already knows the correct hash. It does **not** cover rollback, indefinite freeze, mix-and-match, malicious-mirror denial, or graceful recovery from key compromise. [tuf-go-tuf §4] That delta is the case for a metadata/trust framework on top. [tuf-go-tuf §4, §9]

2. Decision drivers

- **Compromise resilience.** The online Go control plane is a single, attractive signing-key target; an attacker who pops it should not be able to ship arbitrary, stale, or device-targeted firmware. [uptane §9] [tuf-go-tuf §4]
- **Minimal-wrapping locked strategy.** Adopt only the trust machinery that adds value over the existing per-artifact signing; avoid embedded-Linux/C++ baggage that mismatches Go+Android. [ota_landscape_report §3.3, §4]
- **Mandated stack fit.** Prefer an idiomatic Go library that publishes over the existing HTTP/3→HTTP/2 + Brotli path and hooks into the MinIO/S3 upload flow. [tuf-go-tuf §5, §6]
- **Android-15-first reality.** The device-side trust client is the dominant cost/risk and is currently unproven. [tuf-go-tuf §7, §8] [uptane §8]
- **Verify-before-apply requirement.** Local-verified-file apply is safer than streaming an unverified payload straight to applyPayload. [additions_synthesis §6]
- **No-guessing / research-before-implementation.** No trust mechanism is made mandatory on-device without a spike that resolves the open client path. [additions_synthesis §5 C5, §7]

3. Options considered

3.1 Option A — Plain per-artifact signing only (SHA-256 + detached signature + AVB)

Keep what both drafts already converged on: SHA-256 + detached signature verified server-side on upload and device-side before apply, plus Android Verified Boot (AVB) and the A/B engine's own payload check. [additions_synthesis §3, §5 C5]

- **Pros:** Zero new dependency; already in scope; native A/B provides atomic apply + automatic boot-failure rollback independently of any metadata layer. [ota_landscape_report §3.1] [uptane §9]
- **Cons:** Does **not** mitigate rollback, fast-forward, indefinite-freeze, mix-and-match, malicious-mirror, or key-compromise-recovery attacks; security collapses to a single signing key. [tuf-go-tuf §4]
- **Verdict:** Necessary but **insufficient as the end state**. Correct as the MVP floor and retained as defense-in-depth regardless of later layers. [tuf-go-tuf §9] [additions_synthesis §5 C5]

3.2 Option B — TUF via go-tuf/v2 server-side, device enforcement gated

Adopt TUF (CNCF-graduated spec) using `github.com/theupdateframework/go-tuf/v2` as the server-side metadata/trust layer; gate *mandatory* device-side enforcement behind a dedicated Android-client spike. [tuf-go-tuf §9] [ota_landscape_report §3.3]

Verified library facts (GitHub API + pkg.go.dev, 2026-06-07): module path `github.com/theupdateframework/go-tuf/v2`; license **Apache-2.0**; latest release **v2.4.2 (2026-05-19)**; pinned spec version **1.0.31**; crypto ED25519/RSA/ECDSA; consistent snapshots, standard + hash-bin delegations, succinct hash-bin (TAP-15), multi-repo consensus (TAP-4); generics-based API. [tuf-go-tuf §5] Adopter list is the project's own claim — **UNVERIFIED** for any specific adopter. [tuf-go-tuf §5]

- **Additive/non-invasive layering:** artifacts (`payload.bin` / `.zip`) remain byte-identical and keep their AOSP/payload signature; TUF treats each as an opaque target file (path + length + SHA-256/512). The publish path computes `TargetFiles.FromFile(...)`, bumps snapshot + timestamp, signs, and writes metadata next to the blobs; served over the existing HTTP/3→HTTP/2 + Brotli path. [tuf-go-tuf §6]
- **Attack classes closed over plain signing:** arbitrary-install, rollback, fast-forward, indefinite-freeze, endless-data, extraneous-dependencies, mix-and-match, wrong-software, malicious-mirror denial, and key-compromise resilience via thresholds + offline keys. [tuf-go-tuf §4]
- **Effort (estimates, UNVERIFIED until spiked):** server-side LOW–MEDIUM (~1–2 engineer-weeks for first integration + tests); key management MEDIUM–HIGH (process: offline root/targets custody, signing ceremony, threshold + rotation, timestamp re-signing automation); device-side **HIGH** / **OPEN**. [tuf-go-tuf §7]
- **The blocker is device-side, not the library:** no verified production-grade Android/Kotlin TUF client exists; on-device path is gomobile/JNI vs hand-rolled Kotlin vs an existing JVM client (**none verified to exist or be**

maintained) — all **UNVERIFIED**. [tuf-go-tuf §7, §8] In the landscape matrix this is the dominant adoption risk and pins tuf-go-tuf Android-15 fit at 2/5 with an **UNVERIFIED** client. [ota_landscape_report §2.3]

- **metadata/updater and metadata/trustedmetadata client-flow API signatures** were not fully enumerated from pkg.go.dev (truncated) — **UNVERIFIED**; confirm before sizing client work. [tuf-go-tuf §5, §8]
- **Verdict: Adopt server-side now; gate device-side enforcement behind a spike.** Strong Go fit (5/5), Apache-2.0, actively maintained; net wrap-ability high on server, blocked on device only by the missing client. [ota_landscape_report §2.3, §3.3] [tuf-go-tuf §9]

3.3 Option C — Full Uptane (dual-repo + Primary/Secondary) now

Adopt the full Uptane standard (2.1.0): dual Image + Director repositories, four TUF roles per repo, and the Primary/Secondary ECU verification topology. [uptane §3, §4, §5, §6]

- **Genuinely valuable to borrow:** dual-repository (Director + Image) compromise resilience — security degrades only if **both** repos are compromised; the TUF role model with threshold/offline keys; per-device Director targeting + inventory DB (maps onto staged rollout 5/10/30/.../100%); full-verification semantics as a testable Android security bar. [uptane §7, §9]
- **Does not fit / overhead now:** the Primary/Secondary ECU tier is **vacuous** for a fleet of independent, full-power Android devices (no in-vehicle bus, no constrained downstream MCU) — each Helix device == a Primary doing full verification. [uptane §6, §9] No first-class AOSP Uptane client; any Android+Uptane integration is **bespoke**, and it is **UNVERIFIED that any production Android+Uptane stack ships today**. [uptane §8, §9] No authoritative current Rust (“uptane-rs”) reference implementation found — **UNVERIFIED**, do not assume one exists. [uptane §8]
- **Spec vs implementation maturity:** spec maturity is high (IEEE-ISTO/Linux-Foundation governed; Standard 2.1.0 — exact publication date **UNVERIFIED**); implementation cadence is low. [uptane §3] [ota_landscape_report §2.3]
- **Verdict: Reject as a now-MVP whole-system adoption; adopt the *model* selectively as a 1.0.1+ stretch hardening goal** — implement a Go Director + Image split, skip the Primary/Secondary tier, do not adopt aktualizr. [uptane §10] [ota_landscape_report §3.3, §4] This is the recorded C5 resolution. [additions_synthesis §5 C5]

3.4 Option D — aktualizr as a product

Use the primary Uptane reference client, aktualizr (C++, MPL-2.0). [uptane §8]

- **Cons:** C++/Yocto/OSTree-coupled (mismatched with Go + Android update_engine); latest tagged release **Feb 2020** (low cadence); no Go reference impl; no first-class Android integration. [uptane §8] In the landscape this drives Uptane’s wrap-ability to 2/5 (low as whole-system wrap; high only as a borrowable design). [ota_landscape_report §2.3]
- **Verdict: Reject** for adoption/wrapping; retain the security *model* only. [uptane §10] [ota_landscape_report §4]

4. Decision

Adopt a phased trust model. MVP (1.0.0) ships plain per-artifact trust (signed artifact + SHA-256 + AVB). TUF (go-tuf/v2) is adopted server-side as the metadata/trust layer in 1.0.1+, with mandatory device-side enforcement gated behind an Android-client spike. The Uptane *dual-repo model* (Director + Image, in Go) is a 1.0.1+ stretch hardening goal; full Uptane and aktualizr are rejected. Signing interfaces are designed MVP-forward so TUF/Uptane drop in without rework. [additions_synthesis §5 C5] [ota_landscape_report §3.3] [tuf-go-tuf §9] [uptane §10]

4.1 MVP trust model (1.0.0)

- Per-release **SHA-256 + detached signature**, verified server-side on upload and **device-side before apply**; plus **AVB** and the A/B engine's own payload-signature check. These remain as defense-in-depth in every later phase. [additions_synthesis §3, §5 C5] [tuf-go-tuf §9]
- **Local-verified-file apply** is chosen over streaming an unverified payload directly to applyPayload. This decision is justified by the **verify-before-apply requirement** (a recorded decision driver), which mandates a fully verified local artifact before apply; streaming an unverified payload would bypass that gate. (Draft 02's file://-to-applyPayload path vs HTTPS-streaming path is an explicit pick the spec must justify; additions_synthesis §6 flags this as a choice to be justified rather than as the deciding authority.) [additions_synthesis §6]

4.2 MVP-forward signing interface (Uptane-ready)

Design the MVP signing/verification interfaces so TUF (then a Director+Image split) drops in without rework: [additions_synthesis §5 C5]

- Treat every artifact as an **opaque target identified by path + length + SHA-256** so a TUF targets entry layers over it byte-identically later (artifacts keep their existing signature). [tuf-go-tuf §6]
- Put signing behind a **signer abstraction** compatible with go-securesystemslib signature.Signer (the integration point go-tuf/v2 uses), so the MVP detached-signature signer and a future TUF role signer share one seam. [tuf-go-tuf §5]
- Keep verification a **distinct device-side step that gates apply**, so a TUF refresh/verify flow (role order root→timestamp→snapshot→targets, expiry, version monotonicity, threshold sigs) can be inserted in front of the existing hash+signature check without changing the apply path. [tuf-go-tuf §3, §6]
- Reserve a **per-device “what should this device install” decision** in the control plane (already required by staged rollout) so a Director repository can later mint per-device targets.json from the inventory DB. [uptane §6, §9]
- Route the security/signing work through the verified catalogue (security, KMP Security-KMP) rather than inventing new bricks. [additions_synthesis §6]

4.3 Adoption sequencing (1.0.1+)

Per the TUF and Uptane notes, in order: [tuf-go-tuf §9] [uptane §10]

1. Prototype the **server publish path + a Go reference refresh client** end-to-end to validate the model. [tuf-go-tuf §9]
2. **ADR + spike for the on-device client** (gomobile/JNI vs hand-rolled Kotlin) — this dominates adoption cost and must resolve the **UNVERIFIED** client gap. [tuf-go-tuf §7, §9]
3. Define the **offline key-custody ceremony** (HSM/KMS, threshold, rotation; timestamp re-signing automation). [tuf-go-tuf §7, §9]
4. Only then make **device-side TUF verification mandatory**. [tuf-go-tuf §9]
5. **Stretch:** implement a TUF/Uptane-inspired **Director (online keys, per-device) + Image (offline-keyed) split in Go**, Android client does **full verification**; skip the Primary/Secondary tier; do **not** adopt aktualizr. [uptane §10] [ota_landscape_report §3.3]

5. Consequences

5.1 Positive

- **Ships a safe MVP without blocking on an unsolved problem.** The unresolved device-side TUF client (the long pole) does not gate 1.0.0; native A/B already guarantees atomic apply + automatic boot-failure rollback. [tuf-go-tuf §8] [ota_landscape_report §3.1]
- **Clear upgrade path with no artifact rework.** TUF is additive/non-invasive — artifacts stay byte-identical, so 1.0.0 releases remain valid targets when TUF lands. [tuf-go-tuf §6]
- **Closes real attack classes in 1.0.1+** (rollback, fast-forward, freeze, mix-and-match, mirror-denial, key-compromise recovery) that plain signing leaves open. [tuf-go-tuf §4]
- **Nation-state-grade compromise resilience available as a stretch** via the dual-repo split, hardening the online Go control plane without an embedded-Linux C++ client. [uptane §9, §10]
- **Strong mandated-stack fit:** go-tuf/v2 is idiomatic Go, Apache-2.0, actively maintained, single dependency tree, serves over the existing HTTP/3→HTTP/2 + Brotli path. [tuf-go-tuf §5, §6] [ota_landscape_report §2.3]

5.2 Negative

- **MVP retains a single-signing-key exposure** (no rollback/freeze/mix-and-match/mirror-denial mitigation until 1.0.1+). [tuf-go-tuf §4] Mitigation: AVB + A/B payload check + server-side upload verification as interim depth. [additions_synthesis §3]
- **Operational key-management burden is inherent to TUF** (offline custody, ceremony, threshold, rotation, timestamp automation) — MEDIUM–HIGH, process not code. [tuf-go-tuf §7]
- **Device-side client is HIGH/OPEN and UNVERIFIED:** no verified production Android/Kotlin TUF client; gomobile adds Go toolchain + APK binary-size cost, hand-rolled Kotlin means owning security-sensitive code. [tuf-go-tuf §7, §8]

- **Unverified API surface:** metadata/updater / metadata/trustedmetadata signatures not fully captured — client sizing is provisional until spiked. [tuf-go-tuf §5, §8]
- **Dual-repo stretch adds operational complexity** (two repos, two key postures) that is only justified if the compromise-resilience requirement is firm. [uptane §4, §9]
- **Board-level unknowns persist** for the device path (Android-15/RK3588 update_engine constants/AIDL headers **UNVERIFIED**), which the trust client must integrate against. [ota_landscape_report §3.1, §5]

6. Status

Proposed. Pending operator review alongside ADR-0001 (engine selection). The 1.0.1+ adoption steps in §4.3 (especially step 2, the on-device client spike) are themselves gated and must close their **UNVERIFIED** items before device-side TUF enforcement becomes mandatory. [tuf-go-tuf §9] [additions_synthesis §7]

7. Compliance notes (HelixConstitution)

Clause text is not duplicated here; clauses are referenced by the numbering and short-labels used across the Helix corpus. The HelixConstitution source file is not present in this repository (**UNVERIFIED** against the source document); labels below are taken from additions_synthesis.md and the master design doc, which cite them. [additions_synthesis §1, §5; 00-master §15]

Clause	Label (per corpus)	How this ADR complies
§11.4.6	No-guessing	Every claim cites a note/report section; unverifiable items carried as UNVERIFIED (device client, adopter list, updater API, Uptane 2.1.0 date, uptane-rs, board specifics), never invented. [tuf-go-tuf §5, §8] [uptane §8, §12]
§11.4.8	Research-before-implementation	Device-side TUF enforcement is gated behind a spike; no on-device trust mechanism is mandated without research evidence. [additions_synthesis §5 C5, §7]
§7.1	No-bluff / evidence-only	MVP-vs-1.0.1 timing decided on evidence; draft marketing claims excluded; positive

Clause	Label (per corpus)	How this ADR complies
		evidence only. [additions_synthesis §6; 00-master §15]
§11.4.74	Catalogue-first reuse	Signing/verify routed through the verified catalogue (security, Security-KMP); no bespoke crypto brick invented where a catalogue brick exists. [additions_synthesis §6]
§11.4.28	Decoupling	Signer abstraction + a distinct verify-gate step keep artifact-validator / signing / update_engine bridge independently testable and Uptane-swappable. [00-master §6 (decoupling principle)]
§1 / §1.1	Four-layer testing + mutation immunity	Safety-critical signing-verify path targets $\geq 90\%$ coverage under the four-layer + mutation regime (source-presence $\rightarrow$ artifact $\rightarrow$ runtime $\rightarrow$ mutation meta-test). [additions_synthesis §5 C8; 00-master §13]
		Unmappable-until-Constitution-present. The HelixConstitution source file is not in this repository, so the exact clause text/requirements of §11.4.123 cannot be verified; a definitive mapping is out-of-scope until the Constitution is available. As a
§11.4.123	Rock-solid-proof	provisional read against the short-label, the spikes in §4.3 (server publish + Go refresh client; on-device client ADR/spike; key-custody ceremony) are intended to replace the carried UNVERIFIED items in §8 with verified evidence before device-side enforcement is mandated

Clause	Label (per corpus)	How this ADR complies
		— but whether that satisfies §11.4.123 is UNVERIFIED against the clause text. [additions_synthesis §5 C5, §7]
§11.4.125	Code-review gate	This ADR is subject to the mandatory adversarial code-review subagent before acceptance. [00-master §14]

8. Open / UNVERIFIED items

Carried forward (must close before device-side TUF enforcement is made mandatory):

1. **Android TUF client** — no verified production Kotlin/JVM client; resolve gomobile/JNI vs hand-rolled Kotlin via spike (dominant adoption cost). **UNVERIFIED.** [tuf-go-tuf §7, §8] [ota_landscape_report §5]
2. **go-tuf/v2 client-flow API** (metadata/updater, metadata/trustedmetadata) signatures not fully enumerated. **UNVERIFIED.** [tuf-go-tuf §5, §8]
3. **go-tuf/v2 production-adopter list** — project’s own claim, not independently audited. **UNVERIFIED.** [tuf-go-tuf §5]
4. **Uptane Standard 2.1.0 publication date** (June 27, 2023) — from listing/secondary metadata, not an in-document string. **UNVERIFIED.** [uptane §3, §12]
5. **Rust (“uptane-rs”) reference implementation** — not found; do not assume it exists. **UNVERIFIED.** [uptane §8, §12]
6. **Any shipping production Android + Uptane integration** — not found; assume bespoke. **UNVERIFIED.** [uptane §8, §12]
7. **Adoption-effort figures** (server ~1–2 engineer-weeks; key-mgmt MEDIUM–HIGH; device HIGH) are estimates, not measured. **UNVERIFIED** until spiked. [tuf-go-tuf §7]
8. **Android-15 / RK3588 update_engine constants/AIDL headers** the trust client integrates against. **UNVERIFIED** (carried from ADR-0001 scope). [ota_landscape_report §3.1, §5]
9. **Catalogue-brick fit for TUF primitives** — the security / Security-KMP catalogue bricks are routed for signing/verify work (§4.2), but their actual fit for TUF-role / threshold / offline-key primitives (i.e. whether they expose or can host these without bespoke crypto) has not been confirmed against the catalogue. **UNVERIFIED.** [additions_synthesis §6]

9. Sources

All paths relative to docs/research/main_specs/research/:

- ota_landscape_report.md — §2.3 (per-criterion scores for tuf-go-tuf, uptane), §3.3 (trust framework recommendation), §4 (what we will not use), §5 (open/UNVERIFIED).
- stacks/tuf-go-tuf.md — §1, §3 (roles), §4 (attack classes), §5 (verified library facts + API surface), §6 (layering), §7 (effort), §8 (risks/gaps), §9 (recommendation).
- stacks/uptane.md — §2–§9 (model, dual-repo, roles, ECU model, threat model, ecosystem, relevance), §10 (recommendation), §12 (confidence/UNVERIFIED notes).
- additions_synthesis.md — §3 (points of agreement), §4 (destinations), §5 C5 + C8 (contradictions/resolutions), §6 (corrections), §7 (routed open questions).
- ../00-master/2026-06-07-helix-ota-design.md — §6 (decoupling), §13 (testing), §14 (code-review gate), §15/§16 (clause mapping, open ADRs).